

DIGITAL ESTATE & FINANCIAL CLOSURE ASSISTANT

PS1 — FinTech Track

IMPLEMENTATION, REPOSITORY REUSE, TEAM WORK SPLIT & API INTEGRATION PLAN

Final execution document based on the finalized PRD and the two reference repositories.

Product	Digital Estate & Financial Closure Assistant for Grieving Families
Format	Mobile-first Progressive Web App (PWA)
Privacy Architecture	TEE / Confidential Computing with protected document processing
Primary Stack	React + TypeScript + Vite; FastAPI; PostgreSQL; encrypted object storage; custom extraction model
Document Status	Final implementation plan

Tagline: “Don’t leave your family a scavenger hunt.”

Table of Contents

1. Purpose and Final Implementation Direction
2. Final Repository / Folder Structure
3. Repository Reuse Plan
4. Files to Reuse, Rewrite, Reference, or Exclude
5. Frontend Work Split — Two People
6. Backend Work Split — Two People
7. API Contract and Endpoint Paths
8. Frontend–Backend Connection Contract
9. End-to-End Data and Processing Flow
10. Git / Branch / Integration Workflow
11. Recommended Implementation Order
12. Definition of Done
13. Source References and Important Constraints

1. Purpose and Final Implementation Direction

This document converts the finalized Digital Estate & Financial Closure Assistant PRD into a practical engineering plan for a four-person hackathon team. It defines the target folder structure, the exact reuse strategy for the two reference repositories, the frontend/backend work split, and the shared API contract so that all four contributors can work in parallel without creating integration ambiguity.

The product remains a privacy-first digital estate and financial closure assistant. It combines a secure legacy vault with a post-death estate workflow that turns scattered financial documents into structured information, an estate inventory, and a prioritized action plan.

1.1 Fixed architectural decisions

- No blockchain, Algorand, ASA, NFT, smart-contract, wallet-address, or on-chain ownership-transfer functionality.
- No automatic legal transfer of assets or ownership.
- No DigiLocker, Aadhaar, or UPI dependency in the MVP.
- Sensitive source documents should not be sent to a conventional external general-purpose LLM API by default.
- TEE / confidential computing is the protected processing boundary; the custom extraction model runs inside that boundary.
- The product is a responsive, installable PWA with a mobile-first experience.
- The application receives structured extraction results rather than unnecessary raw document content.
- Extracted information is treated as unconfirmed until validated when confidence or business impact requires human review.

1.2 Product flow

```
Owner prepares vault
↓
Trusted family / beneficiary access
↓
Estate case created
↓
Family uploads mixed financial documents
↓
Secure / TEE-protected processing
↓
OCR + classification + custom extraction model
↓
Validation + confidence + source region
↓
Human confirmation where required
↓
Estate inventory
↓
Action engine + explainable priority
↓
Tasks + deadlines + dependencies
↓
Audit trail and family collaboration
```

2. Final Repository / Folder Structure

The following structure is the target implementation repository. The structure deliberately separates the user-facing application, API/business logic, custom extraction pipeline, TEE worker, database migrations, tests, and documentation.

```
digital-estate-closure/
├──
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   │   ├── ui/
│   │   │   └── layout/
│   │   └── pages/
│   │       ├── Login.tsx
│   │       ├── Register.tsx
│   │       ├── Home.tsx
│   │       ├── Vault.tsx
│   │       ├── Documents.tsx
│   │       ├── Estate.tsx
│   │       ├── Actions.tsx
│   │       ├── Family.tsx
│   │       ├── Audit.tsx
│   │       └── Settings.tsx
│   ├── vault/
│   ├── documents/
│   ├── extraction/
│   ├── estate/
│   ├── actions/
│   ├── family/
│   ├── audit/
│   └── services/
│       ├── api.ts
│       ├── authApi.ts
│       ├── vaultApi.ts
│       ├── documentApi.ts
│       ├── extractionApi.ts
│       ├── estateApi.ts
│       ├── actionApi.ts
│       └── familyApi.ts
```

```

■ ■ ■ ■■■ auditApi.ts
■ ■ ■■■ hooks/
■ ■ ■■■ types/
■ ■ ■■■ utils/
■ ■ ■■■ App.tsx
■ ■ ■■■ main.tsx
■ ■ ■■■ index.css
■ ■■■ package.json
■ ■■■ vite.config.ts
■ ■■■ tsconfig.json
■ ■■■ .env.example
■
■■■ backend/
■ ■■■ app/
■ ■ ■■■ main.py
■ ■ ■■■ api/
■ ■ ■ ■■■ auth.py
■ ■ ■ ■■■ vault.py
■ ■ ■ ■■■ documents.py
■ ■ ■ ■■■ extraction.py
■ ■ ■ ■■■ estate.py
■ ■ ■ ■■■ assets.py
■ ■ ■ ■■■ liabilities.py
■ ■ ■ ■■■ actions.py
■ ■ ■ ■■■ tasks.py
■ ■ ■ ■■■ family.py
■ ■ ■ ■■■ audit.py
■ ■ ■ ■■■ notifications.py
■ ■ ■■■ models/
■ ■ ■■■ schemas/
■ ■ ■■■ services/
■ ■ ■ ■■■ vault_service.py
■ ■ ■ ■■■ document_service.py
■ ■ ■ ■■■ extraction_service.py
■ ■ ■ ■■■ estate_service.py
■ ■ ■ ■■■ action_engine.py
■ ■ ■ ■■■ priority_engine.py
■ ■ ■ ■■■ task_service.py
■ ■ ■ ■■■ family_service.py
■ ■ ■ ■■■ permission_service.py
■ ■ ■ ■■■ notification_service.py
■ ■ ■ ■■■ audit_service.py
■ ■ ■■■ security/
■ ■ ■ ■■■ auth.py
■ ■ ■ ■■■ encryption.py
■ ■ ■ ■■■ permissions.py
■ ■ ■ ■■■ audit.py
■ ■ ■■■ repositories/
■ ■ ■■■ core/
■ ■ ■■■ config.py
■ ■ ■■■ database.py
■ ■■■ workers/
■ ■■■ requirements.txt
■ ■■■ .env.example
■
■■■ extraction/
■ ■■■ preprocessing/
■ ■■■ ocr/
■ ■■■ classification/
■ ■■■ models/
■ ■■■ schemas/
■ ■■■ inference/
■ ■■■ validation/
■ ■■■ tests/
■
■■■ tee/
■ ■■■ worker/
■ ■■■ crypto/
■ ■■■ attestation/
■ ■■■ key_release/

```

```

■   ■■■ runtime/
■   ■■■ tests/
■
■■■ database/
■   ■■■ migrations/
■   ■■■ seed/
■
■■■ tests/
■   ■■■ frontend/
■   ■■■ backend/
■   ■■■ extraction/
■   ■■■ security/
■   ■■■ integration/
■
■■■ docs/
■   ■■■ architecture.md
■   ■■■ api.md
■   ■■■ extraction.md
■   ■■■ security.md
■
■■■ .gitignore
■■■ docker-compose.yml
■■■ README.md
■■■ LICENSE

```

3. Repository Reuse Plan

3.1 Legacy Vault repository

Reference repository: <https://github.com/slavhate/legacy-vault>

Use this repository primarily for vault, access-control, beneficiary, dead-man's-switch, encryption, authentication, audit, and basic responsive UI concepts. The implementation should be ported into the new architecture rather than copied wholesale.

Reference file	Target location	Treatment	Purpose
server/crypto.js	backend/app/security/encryption.py	Rewrite	Port encryption concepts to Python and production-oriented key handling.
server/config.js	backend/app/core/config.py	Rewrite	Centralize environment and security configuration.
server/db.js	backend/app/core/database.py + database/migrations/	Rewrite	Move from the original database approach to PostgreSQL.
server/middleware/auth.js	backend/app/security/auth.py	Rewrite	Port authentication middleware to FastAPI.
server/routes/vault.js	backend/app/api/vault.py + vault_service.py	Rewrite	Reuse vault API concepts and CRUD behavior.
server/routes/beneficiaries.js	backend/app/api/family.py + family_service.py + permission_service.py	Rewrite	Convert beneficiary logic into explicit family membership and scoped permissions.
server/routes/deadswitch.js	backend/app/services/legacy-release service	Reference / Rewrite	Retain controlled release concept; do not implement automatic ownership transfer.
public/js/sections.js	frontend/src/vault/ + types/	Adapt	Reuse section/category concepts.

Reference file	Target location	Treatment	Purpose
public/js/api.js	frontend/src/services/api.ts	Rewrite	Create a typed API client using the common /api/v1 base path.

3.2 Digital Estate repository

Reference repository: <https://github.com/deshanashah17/digital-estate.git>

Use this repository mainly for the React + TypeScript + Vite frontend scaffold and selected generic UI patterns. Its blockchain/Algorand architecture is not part of the final product.

Reference file / area	Target location	Treatment	Purpose
frontend/package.json	frontend/package.json	Adapt	Reuse dependency/build direction after removing unrelated dependencies.
frontend/vite.config.ts	frontend/vite.config.ts	Adapt	Reuse Vite configuration as a starting point.
frontend/tsconfig.json	frontend/tsconfig.json	Adapt	Reuse TypeScript project configuration.
frontend/src/main.tsx	frontend/src/main.tsx	Adapt	Reuse application bootstrap pattern.
frontend/src/components/	frontend/src/components/	Selective reuse	Reuse only generic UI patterns that fit the final UX.
frontend/src/App.tsx	frontend/src/App.tsx	Rewrite	Replace blockchain-oriented routing with final product routes.
agents/estate_agent.py	backend/app/main.py / services/	Reference only	Use only for understanding prior service/API structure.
vault/asset_vault.py	backend/app/services/ + models/	Reference only	Do not copy blockchain-dependent asset logic.
agents/sentinel_agent.py	None	Exclude	Not required by the finalized architecture.
agents/nominee_agent.py	None	Exclude	Do not encode automatic nominee ownership/transfer behavior.
contracts/ + *.teal	None	Exclude	Blockchain/smart-contract functionality is explicitly out of scope.
integrations/digilocker.py	None for MVP	Exclude	Not a core MVP integration.
integrations/aadhaar_kyc.py	None for MVP	Exclude	Not a core MVP integration.
integrations/upi_settlement.py	None for MVP	Exclude	Not a core MVP integration.

4. Files to Reuse, Rewrite, Reference, or Exclude

4.1 Reuse philosophy

Do not merge the two repositories directory-to-directory. Treat them as two source code references with different strengths. Legacy Vault supplies the strongest reference for secure vault and access concepts. Digital Estate supplies the strongest starting point for the modern React/Vite frontend scaffold and some FastAPI-era architecture context.

4.2 What must be newly built

- TEE/confidential-processing worker and its protected document-processing boundary.
- Custom document classification and extraction model pipeline.
- Confidence scoring, source-region tracking, and human confirmation flow.
- Estate inventory that unifies extracted and manually confirmed entities.
- Action engine that converts estate entities into actionable checklists.
- Explainable priority and timeline engine.
- Family task assignment and evidence tracking.
- New PostgreSQL data model for estate state.
- Typed `/api/v1` frontend service layer.

4.3 What must not be copied

- Algorand SDK usage, wallet handling, transaction IDs, ALGO/ASA logic, smart contracts, TEAL files, or on-chain asset transfer.
- Blockchain-based will/ownership execution.
- Automatic legal transfer logic.
- Old DigiLocker/Aadhaar/UPI integrations as MVP dependencies.
- Any frontend secrets or private keys.

5. Frontend Work Split — Two People

5.1 Frontend Person A — Vault, Authentication, Family Access, PWA Shell

Area	Primary files	Responsibilities	Depends on backend
App shell	App.tsx, main.tsx, layout/	Routing, navigation, responsive shell, PWA installability.	Auth/session endpoints
Authentication	pages/Login.tsx, Register.tsx, services/authApi.ts	Register, login, current user, logout/session handling.	/auth/*
Vault	pages/Vault.tsx, vault/, services/vaultApi.ts	Vault sections, entry CRUD, protected views.	/vault/*
Family	pages/Family.tsx, family/	Members, permissions, beneficiary/trusted-person access.	/estate/{id}/members, /members/{id}/permissions
Security UI	audit/ + shared components	Access visibility and security status cards.	Audit endpoints
PWA	public/, vite.config.ts	Manifest, app shell, installable experience, camera-friendly upload hooks.	None initially

Frontend A acceptance

- A user can register, log in, and maintain an authenticated session.
- A user can view/create/edit/delete vault entries.
- A user can add family members and view permission state.
- The navigation matches Home, Estate, Vault, Documents, Actions, Family, Security/Audit, Settings.
- The PWA works responsively on mobile and desktop.

5.2 Frontend Person B — Documents, Extraction, Estate, Actions, Tasks

Area	Primary files	Responsibilities	Depends on backend
Documents	pages/Documents.tsx, documents/	Batch upload, document list, processing status.	/documents/*
Extraction	extraction/, services/extractionApi.ts	Job progress, extracted fields, confidence, confirmation UI.	/extraction/*
Estate	pages/Estate.tsx, estate/	Assets, liabilities, insurance, retirement, property, obligations.	/estate/*, /assets/*, /liabilities/*
Actions	pages/Actions.tsx, actions/	Recommended actions, priority reasons, deadlines, dependencies.	/actions/*
Tasks	actions/ or family/	Assign tasks, update status, show evidence/dependencies.	/tasks/*
Audit	pages/Audit.tsx	Timeline of sensitive access and workflow events.	/audit/*

Frontend B acceptance

- Family can upload multiple documents and see processing states.
- Low-confidence or high-impact extracted fields can be confirmed.
- Estate inventory updates after confirmed extraction.
- Actions display title, priority, reason, due date, status, and required documents.
- Tasks can be assigned and status can be updated.

6. Backend Work Split — Two People

6.1 Backend Person A — Core API, Database, Vault, Family, Permissions, Audit

Area	Primary files	Responsibilities
FastAPI app	backend/app/main.py	Application bootstrap, routers, middleware, health.
Auth	api/auth.py, security/auth.py	Registration, login, current-user session/token handling.
Database	core/database.py, models/, repositories/, migrations/	PostgreSQL schema, repositories, transactions.
Vault	api/vault.py, vault_service.py	Vault CRUD, protected entries, ownership.
Family	api/family.py, family_service.py	Members, beneficiaries/trusted people, membership.
Permissions	security/permissions.py, permission_service.py	Section-level least-privilege authorization.
Estate base	api/estate.py, estate_service.py	Estate case lifecycle and summary.
Audit	api/audit.py, audit_service.py	Security/workflow event logging and retrieval.
Notifications	api/notifications.py, notification_service.py	In-app notification state.

Backend A acceptance

- All core API routers are mounted under /api/v1.
- PostgreSQL migrations can create the required MVP schema.
- Permission checks are enforced server-side.
- Sensitive actions create audit events.
- The API exposes stable JSON schemas for frontend integration.

6.2 Backend Person B — TEE, Extraction, Estate Intelligence, Actions, Tasks

Area	Primary files	Responsibilities
Documents	api/documents.py, document_service.py	Secure upload metadata, object storage references, document lifecycle.
TEE	tee/worker/, tee/attestation/, tee/key_release/	Protected processing boundary; attestation-ready design.
Extraction	extraction/ + api/extraction.py	OCR/layout, classification, custom model inference, validation, confidence.
Estate intelligence	api/assets.py, api/liabilities.py	Structured asset/liability records from confirmed information.
Action engine	services/action_engine.py	Entity-to-action templates, requirements, dependencies.
Priority	services/priority_engine.py	Explainable priority based on urgency, impact, dependency, recurring loss, missing docs, etc.
Tasks	api/tasks.py, task_service.py	Assignment, status, due dates, evidence.

Backend B acceptance

- A document can be submitted to a processing job.
- Processing returns structured extraction with confidence and source-region metadata.
- Human confirmation can promote extracted data into the estate inventory.
- The action engine can generate loan, insurance, bank, investment, retirement, property, and subscription actions.
- Priority explanations are returned with every recommended action.
- Tasks can be generated/assigned from actions.

7. API Contract and Endpoint Paths

The backend must expose one consistent API namespace: **/api/v1**. The frontend must never hard-code separate route conventions per module. All services call the same base URL and append the endpoint path shown below.

7.1 Authentication

Method	Path	Purpose
POST	/api/v1/auth/register	Create user account.
POST	/api/v1/auth/login	Authenticate user.
GET	/api/v1/auth/me	Return current authenticated user.

7.2 Vault

Method	Path	Purpose
GET	/api/v1/vault/entries	List vault entries visible to current user.
POST	/api/v1/vault/entries	Create vault entry.
PUT	/api/v1/vault/entries/{entry_id}	Update vault entry.
DELETE	/api/v1/vault/entries/{entry_id}	Delete vault entry.

7.3 Documents and extraction

Method	Path	Purpose
POST	/api/v1/documents	Upload/register a document for processing.
GET	/api/v1/documents/{document_id}	Get document metadata and state.
GET	/api/v1/documents/{document_id}/status	Get processing status.
POST	/api/v1/extraction/jobs	Create an extraction job.
GET	/api/v1/extraction/jobs/{job_id}	Get extraction job and results.
POST	/api/v1/extraction/{extraction_id}/confirm	Confirm or correct extracted information.

7.4 Estate, assets and liabilities

Method	Path	Purpose
POST	/api/v1/estate	Create an estate case.
GET	/api/v1/estate/{estate_id}	Get estate case.
GET	/api/v1/estate/{estate_id}/summary	Get dashboard summary.
GET	/api/v1/estate/{estate_id}/assets	List assets.
POST	/api/v1/estate/{estate_id}/assets	Create asset.
PUT	/api/v1/estate/{estate_id}/assets/{asset_id}	Update asset.
GET	/api/v1/estate/{estate_id}/liabilities	List liabilities.
POST	/api/v1/estate/{estate_id}/liabilities	Create liability.
PUT	/api/v1/estate/{estate_id}/liabilities/{liability_id}	Update liability.

7.5 Actions and tasks

Method	Path	Purpose
GET	/api/v1/estate/{estate_id}/actions	List recommended actions.
POST	/api/v1/estate/{estate_id}/actions	Create/manual action.
PUT	/api/v1/estate/{estate_id}/actions/{action_id}	Update action.
POST	/api/v1/actions/{action_id}/assign	Assign action/task to family member.
PUT	/api/v1/tasks/{task_id}/status	Update task status.

7.6 Family, audit and notifications

Method	Path	Purpose
GET	/api/v1/estate/{estate_id}/members	List estate members.
POST	/api/v1/estate/{estate_id}/members	Add member.
PUT	/api/v1/members/{member_id}/permissions	Update section-level permissions.
GET	/api/v1/estate/{estate_id}/audit	Get audit events.
GET	/api/v1/notifications	List notifications.
PUT	/api/v1/notifications/{notification_id}/read	Mark notification as read.

8. Frontend–Backend Connection Contract

8.1 Common environment

```
Frontend .env
VITE_API_URL=http://localhost:8000
```

```
Backend .env
DATABASE_URL=postgresql://...
JWT_SECRET=...
ENCRYPTION_KEY=...
OBJECT_STORAGE_ENDPOINT=...
OBJECT_STORAGE_BUCKET=...
TEE_ENABLED=true
TEE_MODE=development
EXTRACTION_MODEL_VERSION=v1
```

8.2 Common API client

Create exactly one shared HTTP client at `frontend/src/services/api.ts`. All domain services import that client. This prevents every contributor from inventing different base URLs, authentication handling, or error behavior.

```
frontend/src/services/api.ts

const API_URL = import.meta.env.VITE_API_URL;

export async function apiFetch(path: string, options: RequestInit = {}) {
  const response = await fetch(`${API_URL}/api/v1${path}`, {
    ...options,
    headers: {
```

```

    "Content-Type": "application/json",
    ...(options.headers || {})
  },
  credentials: "include"
});

if (!response.ok) {
  throw new Error(await response.text());
}

return response.json();
}

```

8.3 Service examples

```

authApi.ts
register() -> POST /auth/register
login()    -> POST /auth/login
me()       -> GET  /auth/me

documentApi.ts
upload()   -> POST /documents
getStatus() -> GET  /documents/{id}/status

extractionApi.ts
createJob() -> POST /extraction/jobs
getJob()    -> GET  /extraction/jobs/{job_id}
confirm()   -> POST /extraction/{extraction_id}/confirm

estateApi.ts
create()    -> POST /estate
summary()   -> GET  /estate/{estate_id}/summary

actionApi.ts
list()      -> GET  /estate/{estate_id}/actions
update()    -> PUT  /estate/{estate_id}/actions/{action_id}

```

8.4 Response contract principle

- Use stable field names across backend schemas and frontend TypeScript types.
- IDs should be opaque strings/UUIDs; the frontend should not assume database integer IDs.
- Dates should use a consistent ISO-8601 representation.
- Every protected resource must be authorized server-side.
- Errors should use a consistent JSON shape so the frontend can display actionable messages.
- Do not return unnecessary sensitive document contents in ordinary list/detail APIs.

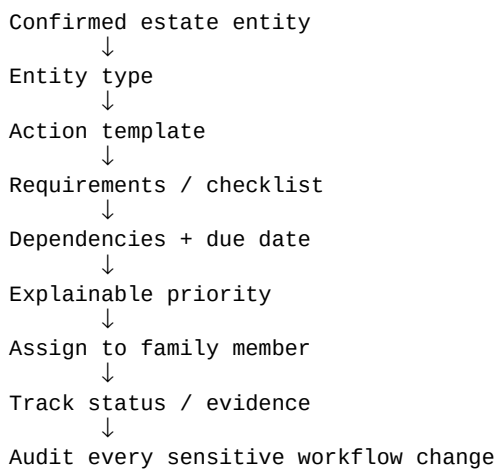
9. End-to-End Data and Processing Flow

9.1 Document flow

1. Frontend sends document metadata/upload request to POST /api/v1/documents.
2. Backend stores the document in encrypted object storage and records metadata in PostgreSQL.
3. Frontend requests POST /api/v1/extraction/jobs.
4. Backend creates a processing job and sends the document reference to the secure processing worker.

5. TEE worker retrieves encrypted content, decrypts only inside the protected processing boundary, performs OCR/layout processing, classification, and custom-model extraction.
6. Extraction output is validated and confidence-scored. Source-region metadata is retained where useful for review.
7. The normal application receives the minimum structured result required for estate workflow.
8. Frontend displays extracted fields and confidence state.
9. User confirms or corrects fields through POST /api/v1/extraction/{extraction_id}/confirm.
10. Confirmed entities are written into the estate inventory.

9.2 Action flow



9.3 Example: loan

A confirmed loan can produce actions such as notifying the lender, collecting the required documents, reviewing applicable coverage, reviewing EMI handling, submitting the required request, and tracking the institution's response. The system recommends next steps; it does not decide legal ownership or automatically transfer the liability.

10. Git / Branch / Integration Workflow

10.1 Recommended branches

Branch	Owner	Scope
main	Team	Stable demo-ready branch.
develop	Team	Integration branch for completed features.
feature/frontend-vault	Frontend A	Auth, vault, family, permissions, PWA shell.
feature/frontend-estate	Frontend B	Documents, extraction UI, estate, actions, tasks, audit.
feature/backend-core	Backend A	Auth, database, vault, family, permissions, audit.
feature/backend-intelligence	Backend B	TEE, extraction, action engine, priority, tasks.

10.2 Shared contract rule

Before implementation, the four developers should freeze the API endpoint names, request payloads, response shapes, IDs, status enums, and error format. Frontend developers can then build against mocked JSON responses while backend work continues.

10.3 Merge order

1. Backend A establishes FastAPI app, PostgreSQL, auth, base models, and /api/v1 router.
2. Frontend A establishes app shell, authentication flow, and shared API client.
3. Backend B establishes document/extraction job schema and processing interfaces.
4. Frontend B builds document and extraction UI against the frozen contract.
5. Backend A and B integrate estate entities and actions.
6. Frontend B connects estate/actions/tasks.
7. Frontend A connects family permissions and audit.
8. Full team runs the golden-path integration test.

11. Recommended Implementation Order

Phase	Work	Owners
Phase 1	Repo setup, folder structure, env files, API contract, database bootstrap.	All / Backend A lead
Phase 2	Auth, app shell, navigation, base PostgreSQL models.	Frontend A + Backend A
Phase 3	Vault, family members, permissions, audit.	Frontend A + Backend A
Phase 4	Document upload, storage metadata, extraction job lifecycle.	Frontend B + Backend B
Phase 5	TEE boundary, OCR/layout, custom extraction model, confidence.	Backend B
Phase 6	Extraction review and confirmation UI.	Frontend B
Phase 7	Estate inventory, assets, liabilities.	Frontend B + Backend A/B
Phase 8	Action engine, priority engine, task assignment.	Backend B + Frontend B
Phase 9	End-to-end integration, security review, audit verification.	All
Phase 10	Golden-path demo rehearsal and deployment.	All

11.1 Golden-path demo

1. Owner creates account.
2. Owner creates/opens vault.
3. Owner adds beneficiary / trusted family member.
4. Owner grants controlled section-level access.
5. Estate case is created.
6. Family member logs in.
7. Family uploads bank + insurance + loan + retirement + property documents.
8. UI shows protected processing / TEE status.
9. Custom extraction produces structured fields.
10. One low-confidence field is shown for human confirmation.
11. Confirmed information appears in estate inventory.
12. Action engine generates next steps.
13. Urgent loan/EMI action is visible with reasons.
14. Action is assigned to a family member.
15. Timeline/status updates are visible.
16. Audit screen shows access and workflow events.

12. Definition of Done

12.1 Product

- A grieving family can move from scattered documents to a prioritized, trackable closure plan.
- The user can distinguish extracted information from confirmed information.
- The system explains why a recommended action is prioritized.
- Family members can be assigned work without receiving unnecessary access.

12.2 Security

- TLS is used for application communication.
- Sensitive data is encrypted at rest.
- Document decryption for extraction occurs only inside the protected processing boundary in the intended deployment.
- Authentication and authorization are enforced server-side.
- No secrets are shipped in the frontend bundle.
- Raw sensitive documents are not written to ordinary application logs.
- Sensitive access and important workflow events are auditable.

12.3 Engineering

- All frontend calls use /api/v1 through the shared API client.
- All backend routers use stable request/response schemas.
- Database migrations can recreate the MVP schema.
- Frontend and backend can run independently using documented environment variables.
- Integration tests cover the golden path.

12.4 TEE wording rule

If the deployed hackathon environment does not provide a real TEE or remote attestation, the team must describe the implementation honestly as a TEE-ready or confidential-processing prototype. Do not claim production-grade hardware isolation when it is not actually present.

13. Source References and Important Constraints

13.1 Finalized PRD

The implementation plan is derived from the finalized Digital Estate & Financial Closure Assistant PRD. The PRD specifies the mobile-first PWA, TEE/confidential-computing processing boundary, custom extraction pipeline, estate inventory, action engine, explainable priority, family collaboration, section-level permissions, auditability, and the /api/v1-style implementation contract.

The finalized PRD also explicitly removes blockchain-based transfer and related functionality from the final MVP direction.

13.2 Reference repositories

Legacy Vault: <https://github.com/slavhate/legacy-vault>

Digital Estate: <https://github.com/deshanashah17/digital-estate.git>

13.3 Final repository decision

Source	Use as	Do not use as
Legacy Vault	Reference for secure vault, authentication, encryption concepts, beneficiaries, access, dead-man's-switch concepts, audit, and basic UI structure.	A direct Node/Express/SQLite codebase to be merged unchanged.
Digital Estate	Reference for React/TypeScript/Vite frontend scaffolding and selected generic UI patterns.	A blockchain/Algorand backend, smart-contract system, or automatic transfer engine.
Final build	New integrated React + FastAPI + PostgreSQL + encrypted object storage + custom extraction + TEE-ready architecture.	A hybrid of conflicting legacy architectures.

13.4 Final engineering principle

Reuse concepts and safe scaffolding; rewrite architecture-sensitive code. The two repositories are source material, not modules that should be blindly combined. The new repository must have one API contract, one database model, one permission model, and one clear document-to-action workflow.

FINAL TEAM CHECKPOINT

Before coding heavily, all four team members should agree on the folder tree, endpoint table, request/response contracts, database entity names, status enums, and the golden-path demo. Once those are frozen, frontend and backend can proceed in parallel with minimal integration friction.